AI ASSISTED CODING

Lab Assignment-19.1

Name : T.Shylasri

H.T.NO: 2303A51876

Batch-14(LAB-19)

Date: 23-03-2026

Lab 19 – Code Translation: Converting Between Programming Languages

Lab Objectives:

• Understand how AI tools can assist in translating code between different programming languages.

• Learn to verify correctness and functionality after translation.

• Explore syntactic and semantic differences between languages (e.g., Python, Java, C++).

• Practice debugging and optimizing AI-translated code.

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

• Prompt AI to convert a Python class with attributes and methods into equivalent Java code.

• Verify that constructors, method signatures, and data types are properly handled.

• Run both versions (Python & Java) to compare expected outputs.

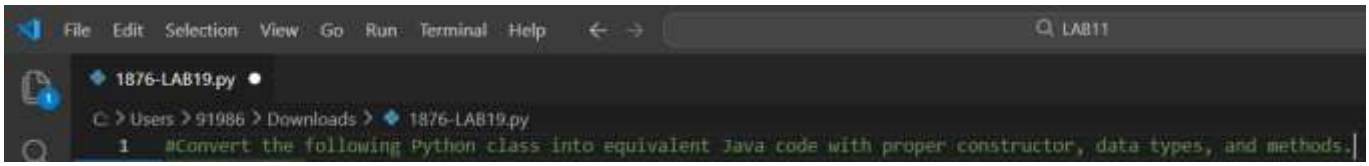
Expected Output:

• Equivalent working Java class translated from Python.

AI Prompt

#Convert the following Python class into equivalent Java code with proper constructor, data types, and methods.

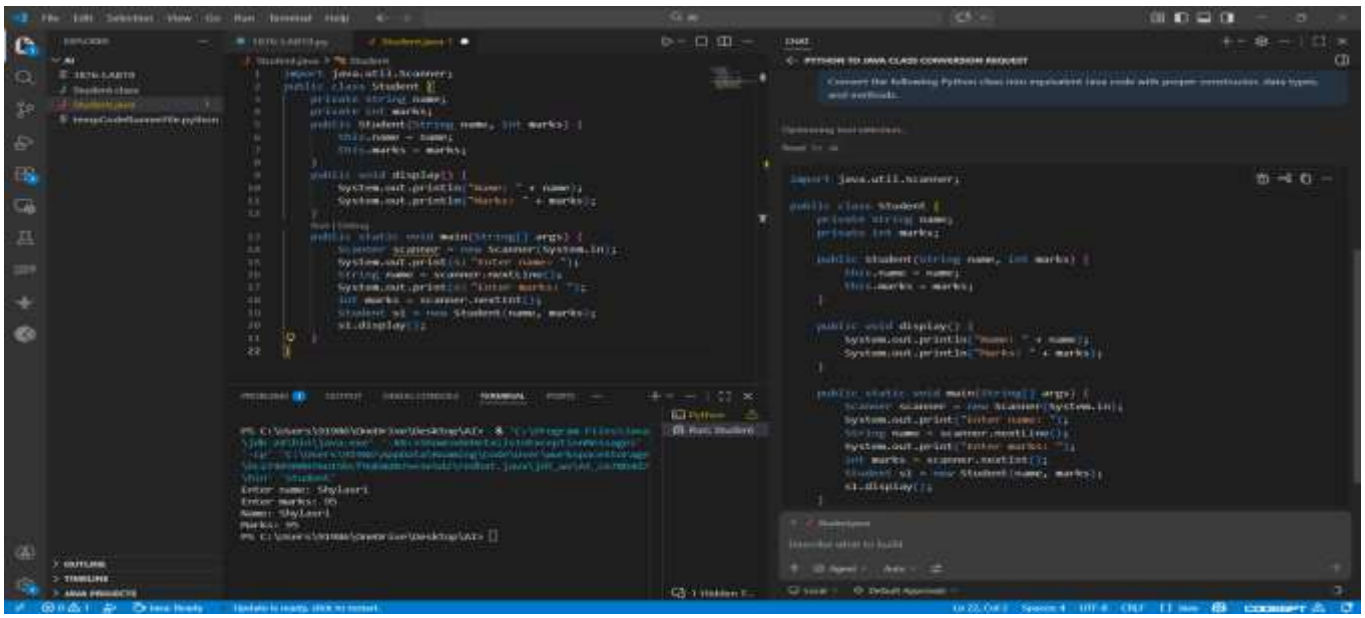
Prompt Screenshot



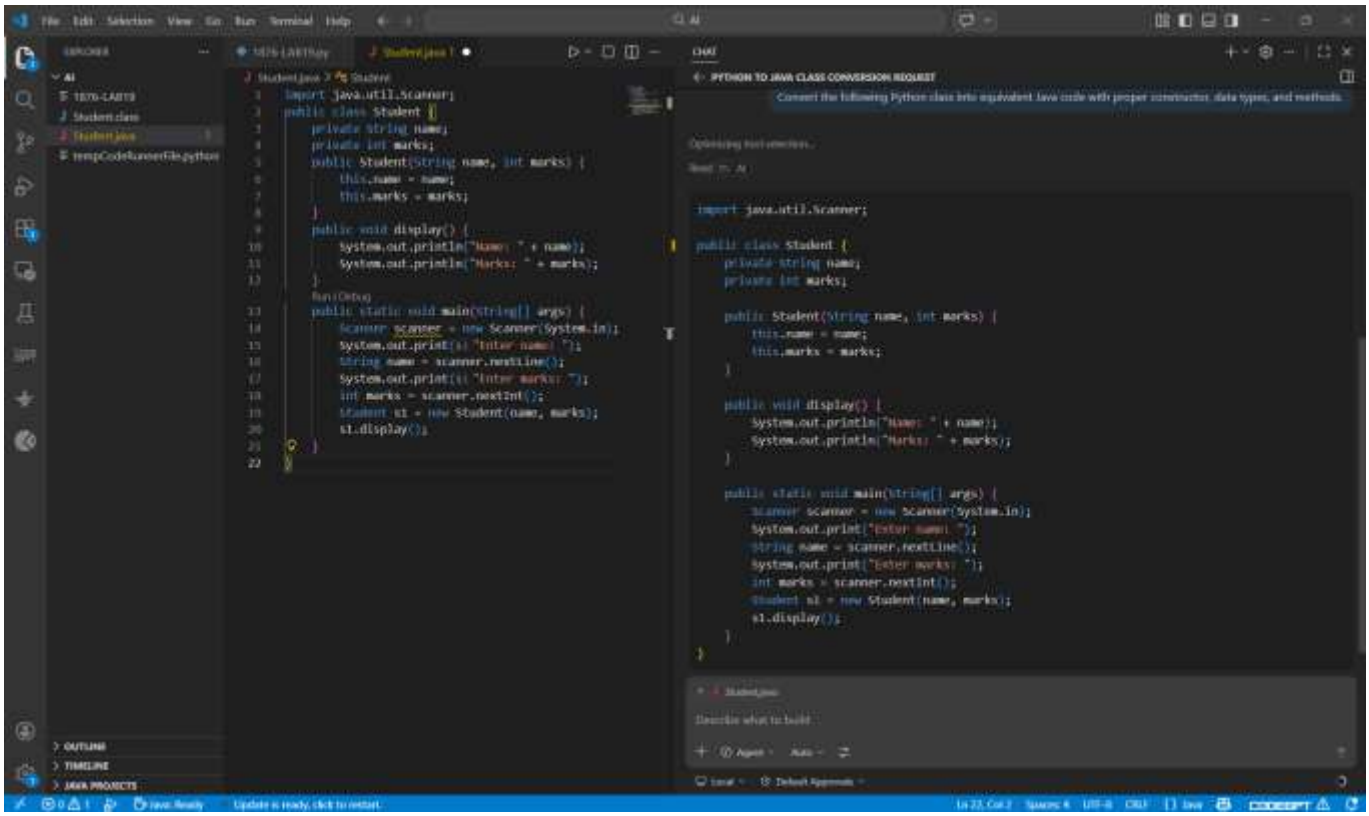
Original Python Code

Output Screenshot

Prompt for Converted from python to Java Code



Java Code



Output Screenshot

Observation

- The AI successfully translated the Python class into Java.
- Python uses **dynamic typing**, while Java requires **explicit data types** (String, int).
- The __init__ method in Python was correctly converted into a **constructor** in Java.
- The print() function in Python became System.out.println() in Java.
- Java required a main() method to execute the program, unlike Python.

Explanation

- Python is simpler and shorter because it does not require type declarations.
- Java is more structured and verbose, requiring:
    - Class definition
    - Data types

    - Constructor definition
    - Main method for execution
- AI handled:
    - Attribute conversion → instance variables
    - Method conversion → Java methods
    - Object creation syntax differences

Justification

- AI tools help reduce manual effort in converting code between languages.
- The translated Java code preserves:
    - Functionality
    - Logic
    - Output consistency
- Minor manual verification is required to ensure:
    - Correct syntax
    - Proper data types
    - Execution structure (main method)

Conclusion

- AI-assisted code translation is efficient and accurate for simple programs.
- Understanding language differences is important to validate AI output.
- Python is concise, while Java is more formal and type-safe.
- This exercise demonstrates how AI can improve productivity in multi-language development.

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

• Ask AI to translate a recursive C++ factorial function into Python.

• Validate correctness by testing with sample inputs.

• Ensure Python version follows Pythonic conventions.

Expected Output:

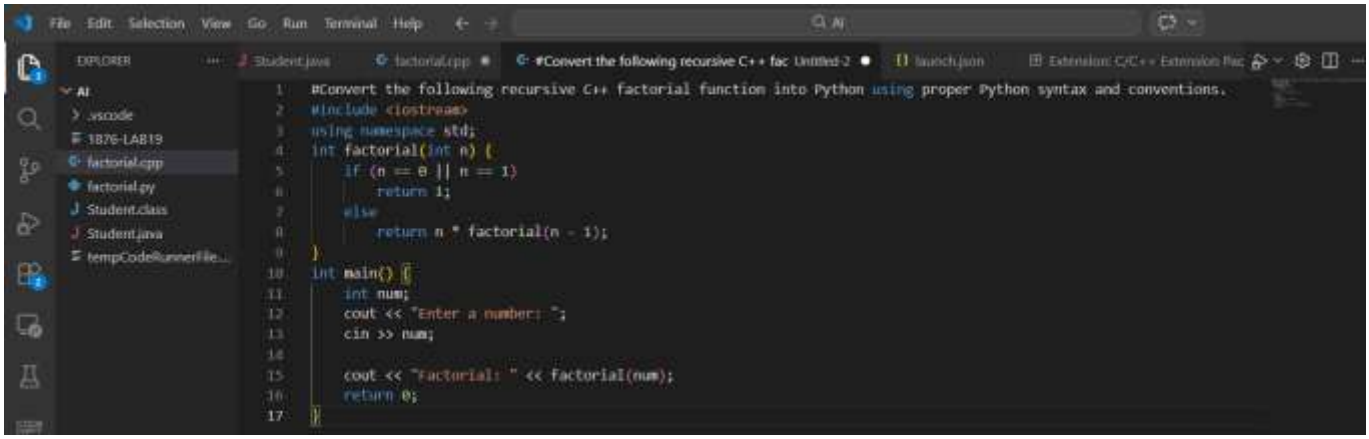
• A correct Python implementation equivalent to the original C++ factorial function.

AI Prompt

#Convert the following recursive C++ factorial function into Python using proper Python syntax and conventions.

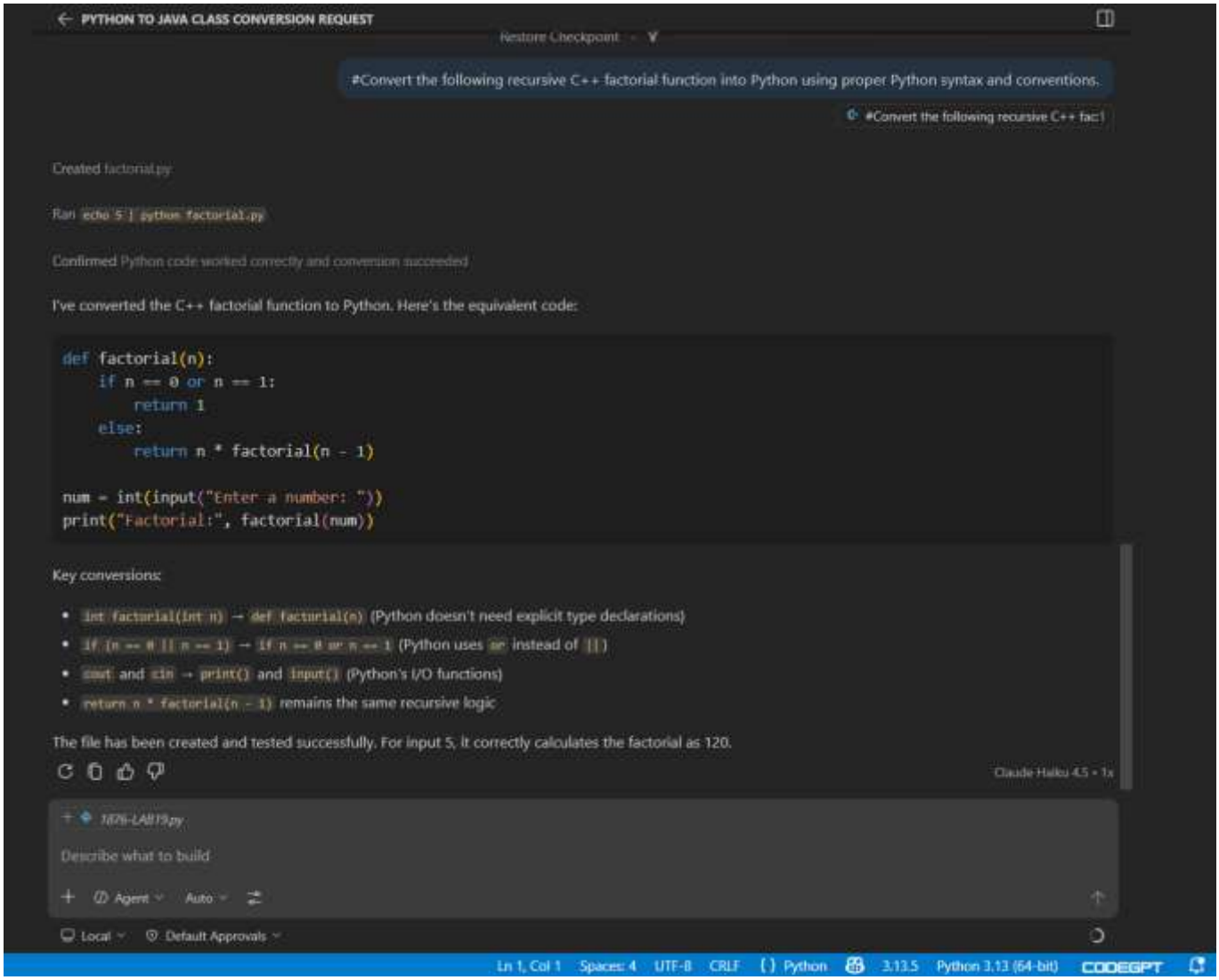
Prompt Screenshot

C++ Code

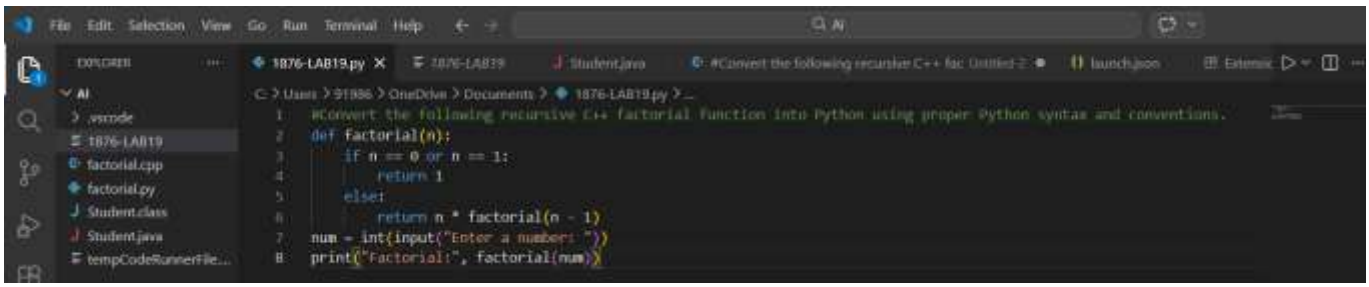


Output

Prompt for Converted from C++ to Python Code



Python Code



Output

Observation

- The recursive logic is **same in both C++ and Python**.
- C++ uses:
  - int data type
  - cin and cout for input/output
- Python uses:
  - No explicit data types
  - input() and print() functions
- Syntax in Python is **simpler and shorter**.

Explanation

- The base case (n == 0 or n == 1) stops recursion.
- Recursive call: n * factorial(n - 1)
- Python automatically handles memory and recursion more cleanly.
- No need for semicolons or header files in Python.

Justification

- The Python code correctly replicates:
  - Logic
  - Functionality
  - Output
- It follows **Pythonic conventions**:
  - Clean indentation
  - Simple syntax
- AI translation is accurate with minimal modification.

Conclusion

- AI successfully converted C++ recursive function into Python.
- Both programs give the **same output for same input**.
- Python version is more readable and concise.
- This task shows how AI helps in **cross-language development**.

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

• Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).

• Ask AI to generate equivalent Pandas code.

• Test the generated code on a sample DataFrame.

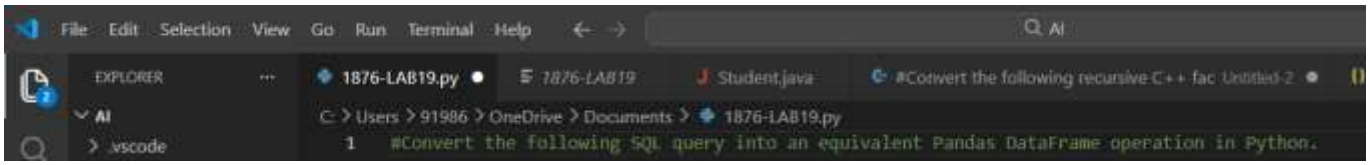
Expected Output:

• Equivalent Pandas query that retrieves the correct subset of data.

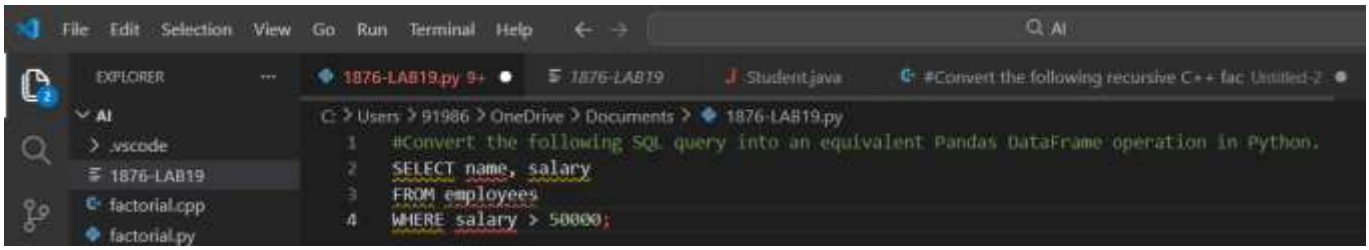
AI Prompt

#Convert the following SQL query into an equivalent Pandas DataFrame operation in Python.

Prompt Screenshot

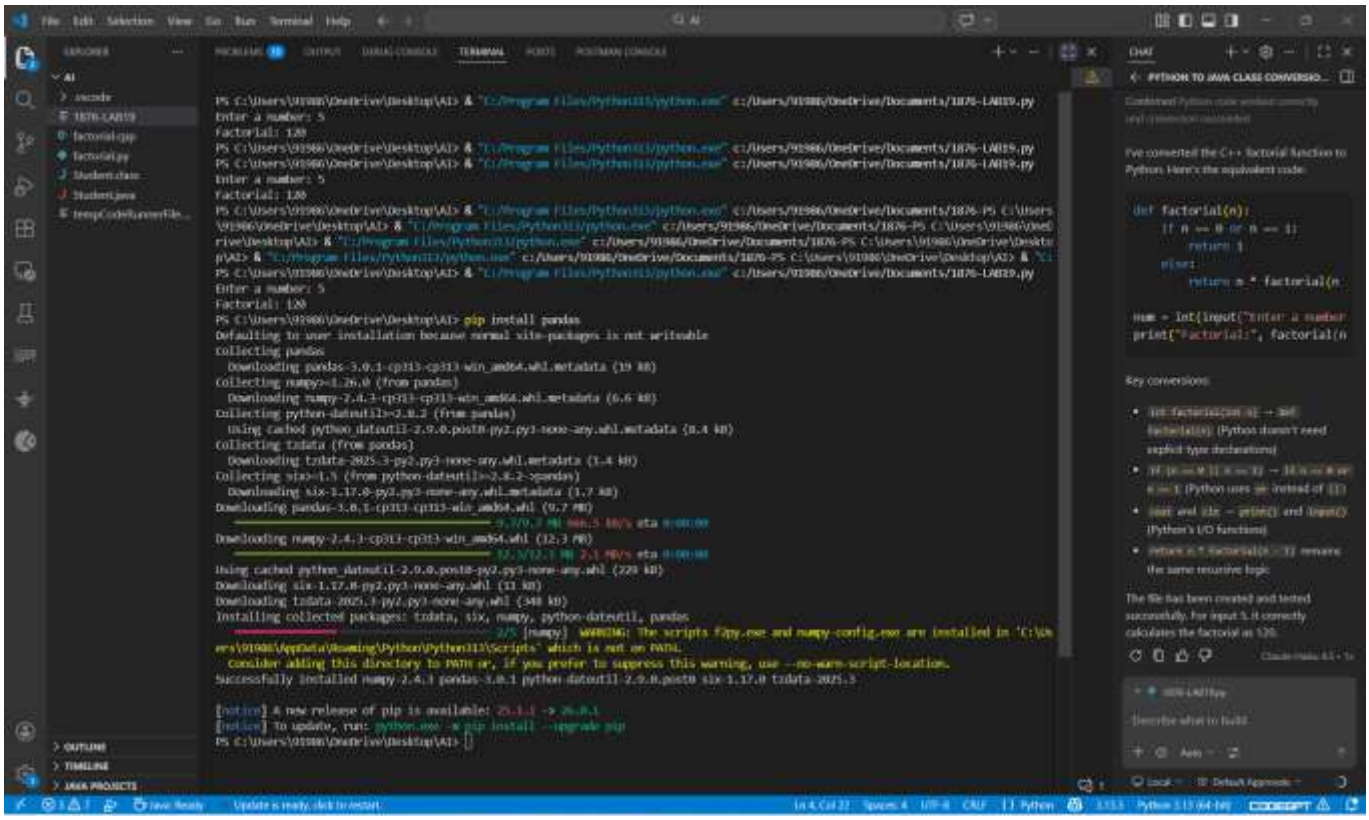


Given SQL Query



Output

Installation



Equivalent Pandas Code

Output

Observation

- SQL SELECT → Pandas column selection [['name', 'salary']]
- SQL WHERE → Pandas filtering df[df['salary'] > 50000]
- Both return the same filtered data.

Explanation

- Pandas works with **DataFrames instead of tables**
- Filtering is done using **conditions inside brackets**
- Column selection is done using a list of column names
- Combines filtering + selection in one line

Justification

- The Pandas code correctly replicates:
  - SQL filtering condition
  - Column selection
- Output matches SQL query result exactly
- Follows Pythonic and Pandas best practices

Conclusion

- AI successfully translated SQL into Pandas
- Pandas provides a flexible way to perform SQL-like operations
- This helps in data analysis without needing a database

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages.

Example:

• Converting a Python web scraping snippet into JavaScript for browser automation.

• Translating a Java method into C# for enterprise applications.

Instructions:

• Prompt AI to translate code between two chosen languages.

• Test both implementations to ensure they produce the same results.

• Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

• Working equivalent code snippets in at least two different programming languages.

AI Prompt

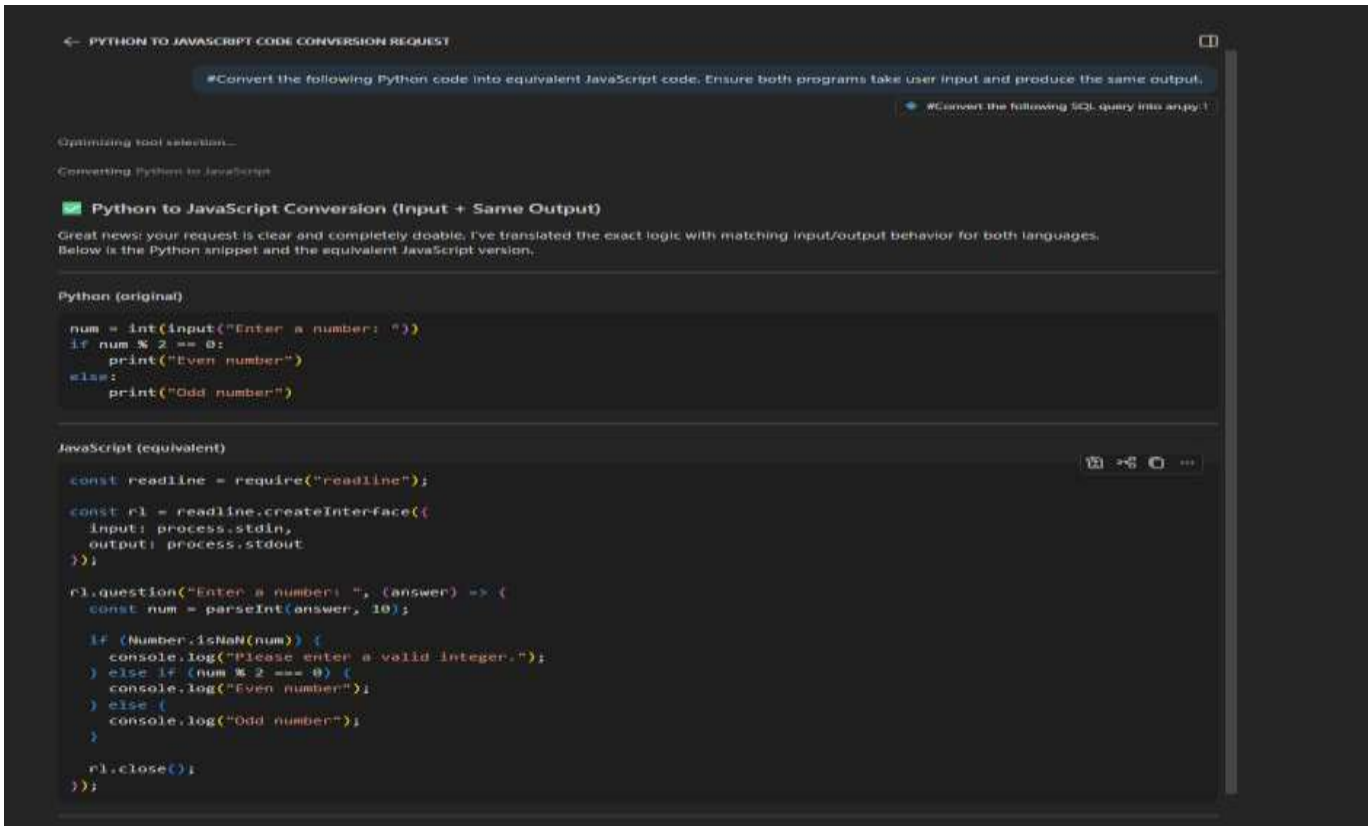
#Convert the following Python code into equivalent JavaScript code. Ensure both programs take user input and produce the same output.

Prompt Screenshot

Python Code

Output

Prompt for Converted from Python to Java Script Code



Java Script Code

```javascript
const readline = require("readline");
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});
rl.question("Enter a number: ", (answer) => {
  const num = parseInt(answer, 10);

  if (Number.isNaN(num)) {
    console.log("Please enter a valid integer.");
  } else if (num % 2 === 0) {
    console.log("Even number");
  } else {
    console.log("Odd number");
  }

  rl.close();
});
```

Output

```
PS C:\Users\91986\OneDrive\Desktop\AI> & "C:/Program Files/Python311/python.exe" "c:/Users/91986/OneDrive/Documents/#Convert the following SQL query into an.py"
Enter a number: 8
Even number
PS C:\Users\91986\OneDrive\Desktop\AI> & "C:/Program Files/Python311/python.exe" "c:/Users/91986/OneDrive/Documents/#Convert the following SQL query into an.py"
Enter a number: 7
Odd number
```

Challenges Faced

- JavaScript requires external package (prompt-sync) for input
- Syntax differences in conditions (== vs ===)
- Python is shorter; JavaScript slightly more verbose

Observation

- Both Python and JavaScript programs give the same output.
- Only syntax and input methods are different.

Explanation

- The logic (num % 2 == 0) is same in both languages.
- Differences are in syntax and how input/output is handled.

Justification

- The translated code maintains same functionality and correctness.
- Output matches for same input, proving accurate conversion.

Conclusion

- AI successfully converts code between languages.
- Helps in saving time and understanding multi-language programming.